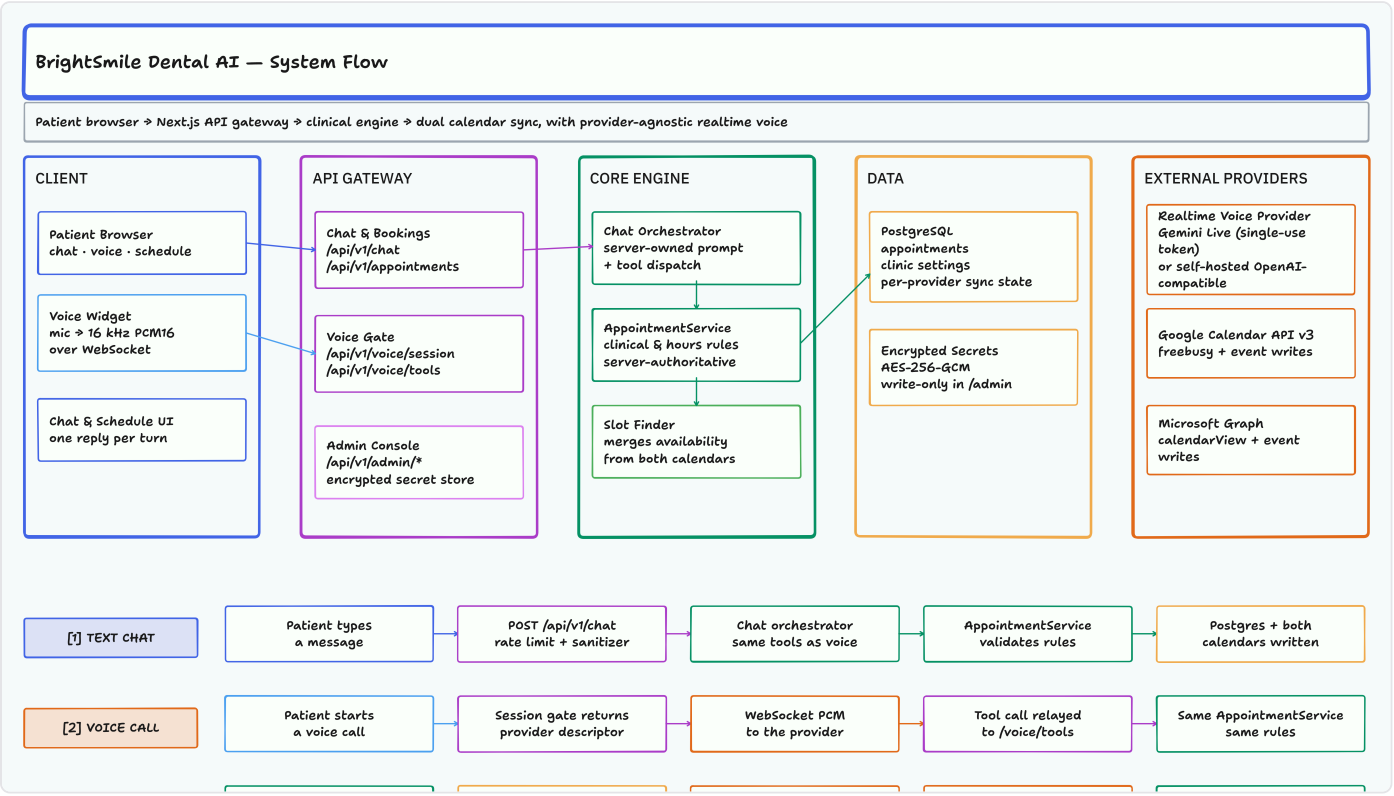
BrightSmile AI: Systems Engineering Specification

System composition, realtime audio mechanics, and the deterministic clinical boundary.

< 300 ms	16 / 24 kHz	30 min	2
TURN LATENCY TARGET	PCM IN / OUT	SLOT QUANTIZATION	VOICE PROVIDERS SHIPPED

System Composition

ARCHITECTURE MAP & RUNTIME LANES



1. Realtime Voice Gateway & Tool Dispatch

WEBSOCKET PCM • PROVIDER-AGNOSTIC

```
// 1. Provider selection + session descriptor (server)
GET  /api/v1/voice/session → { "provider": "gemini-live" | "openai-compatible" }
POST /api/v1/voice/session → 200 OK
{
  "provider": "gemini-live",
  "clientSecret": "auth_tokens/...", // single-use, Gemini only
  "model": "gemini-2.5-flash-native-audio-latest",
  "voice": "Aoede",
  "instructions": "<server-owned system prompt>",
  "tools": [ /* server-owned tool definitions */ ]
}

// 2. WebSocket streaming (browser)
const ws = new WebSocket(descriptor.endpoint);
// PCM16 16 kHz in, 24 kHz out, base64 JSON frames
```

Protocol exchange

1. Selection. `GET /api/v1/voice/session` reports the configured provider so the browser can pick a transport without spending a single-use token.

2. Descriptor. `POST` returns endpoint, model and credentials. With Gemini the server mints a single-use token; with an OpenAI-compatible endpoint there is no credential to hand out, so the role and tools still come from the server.

3. Media. Base64 PCM16 over WebSocket, 16 kHz up and 24 kHz down, no separate STT hop.

4. Tools. The model emits a function call; the browser relays it to `POST /api/v1/voice/tools` for server-side validation, then returns the result.

Providers: `gemini-live` • `openai-compatible` (any Realtime GA endpoint, including a self-hosted **huggingface/speech-to-speech** server)
Tools: `list_services` • `check_availability` • `book_appointment` • `cancel_appointment`

2. Clinical Domain Qualification & Scheduling

DOMAIN BOUNDARY • SRC/DOMAIN/

CODE	PROCEDURE	DURATION	ELIGIBLE	SCOPE
A	**General Checkup & Cleaning**	60 min	`all`	Permitted
B	**Simple Tooth Extraction**	60 min	`all`	Permitted
C	**Root Canal Treatment**	150 min	`senior-1/2`	Specialist
D	**Dental Crown / Veneer**	120 min	`senior-1/2`	Specialist
E	**Full Mouth Implant Surgery**	360 min	`senior-1/2`	Specialist

```
// Clinical authorization guard
if (input.dentistId) {
  const ok = service.eligibleDentistIds
    .includes(input.dentistId);
  if (!ok) throw new DentistQualificationError(
    input.dentistId, service.code);
}
```

30-minute quantization

Clinic hours (09:00–17:00) partition into 30-minute steps. A candidate must fit the whole service (`slot.end ≤ 17:00`) and intersect no busy interval.

Persistence, Calendar Sync & Cost Drivers

Concurrency control, per-provider synchronization state, credential storage, and what the deployment actually costs.

3. Relational Persistence & Concurrency Control

DRIZZLE ORM • POSTGRESQL

```ts
// src/repository/schema.ts (abridged)
export const appointments = pgTable("appointments", {
  id, patientName, patientPhone,
  serviceCode, dentistId,
  date, startTime, endTime,
  status: "pending" | "confirmed" | "cancelled",
});

// one row per appointment + provider
export const appointmentCalendarSyncs = pgTable(..., {
  appointmentId, provider, operation,
  status, eventId, attempts,
  nextAttemptAt, lastError,
});
```

Collision avoidance

1. Availability query. Reads pending and confirmed bookings for the target dentist and date.
2. Provider intersection. Merges database slots with live Google and Microsoft Graph busy intervals.
3. Atomic reservation. Inside a transaction, take a transaction-scoped advisory lock keyed by dentist and date, re-check overlap, then insert a `pending` reservation. The losing transaction gets `SlotUnavailableError` and the assistant offers the next slot.
4. Convergence. The row becomes `confirmed` only after every provider succeeds; retries are claimed with a lease so overlapping runs cannot double-process.

4. Dual-Calendar Adapter Contract

PROVIDER-NEUTRAL ABSTRACTION

Google Calendar

`v3/freeBusy` batch queries. RFC3339 UTC strings with a trailing `Z`. In-memory OAuth token caching with proactive refresh. Booking writes use caller-supplied event IDs and `PUT`, so a retry upserts instead of duplicating.

Microsoft Graph

`calendarView` via Entra ID. Drains multi-page `@odata.nextLink` results. Normalizes `dateTimeTimeZone` payloads to UTC ISO-8601. An ambiguous create is flagged `needs_manual_reconciliation` rather than retried blindly.

Resolution & secrets

`resolveCalendarProvider()` builds both adapters and fails closed: missing credentials or incomplete dentist mappings throw instead of silently skipping a calendar. Only an explicit `CALENDAR_PROVIDER=none` disables external calendars.

5. Cost Drivers

PROVIDER-DEPENDENT, NOT A FIXED MONTHLY TOTAL

COMPONENT	WHAT SETS THE NUMBER
Realtime voice	Hosted providers bill per audio-minute in and out, and this is the only line large enough to dominate a clinic budget. A self-hosted OpenAI Realtime-compatible server turns it into GPU time and electricity.
Text chat LLM	Per input/output token on the configured model. Small next to voice by one to two orders of magnitude.
Hosting	Serverless plan or container, sized to request volume.
PostgreSQL	Managed instance with backups and pooling.
Calendar APIs	Free quotas cover a single clinic at both Google and Microsoft.

Why no total is quoted

A single monthly figure would have to assume a voice provider, an audio-minute volume, and a per-minute rate. Those three assumptions set most of the result, and the native one can be zero. Quoting a total without them is a number, not an estimate.

Price the deployment against the providers you configure. A worked per-minute model with every assumption stated is kept in `docs/INTERNAL_ARCHITECTURE_AND_COST.md`.

6. Managed Cloud vs. Self-Hosted Edge

COST MODEL • LATENCY PROFILE

DIMENSION	MANAGED CLOUD — `GEMINI-LIVE`, HOSTED ENDPOINT	SELF-HOSTED EDGE — `OPENAI-COMPATIBLE` ON LOCAL GPU
Billing unit	Per audio-minute in and out. Every spoken second is metered.	GPU time and electricity. No request is metered or capped.
Latency profile	Vendor inference plus a wide-area round trip from the clinic to a provider region. `< 300 ms` is a modeled target for this class of provider, not a measurement of this deployment.	Silero VAD → streaming STT → LLM → TTS on one local GPU, no vendor hop. Bounded by that GPU's model load and memory, and it degrades with larger models rather than with network distance.
Measurement	Only the vendor reports turn timings; the app sees WebSocket frame arrival.	The `speech-to-speech` server logs per-response STT, LLM and first-TTS-audio durations. That is the only honest source for a latency number.
Credentials	Server mints a single-use ephemeral token per session via `POST /api/v1/voice/session`.	Nothing is minted: there is no metered account to authorize. Role and tools still come from the server in the session descriptor.
Capacity & failure	Provider concurrency quota; an outage is outside the operator's control.	GPU memory, typically one concurrent session per instance; an OOM is a process to restart.
Data path	Audio leaves the clinic network for a vendor region.	Audio stays on the operator's network end to end.

Both columns run the same application code; the choice is configuration (`GEMINI_API_KEY` vs. `OPENAI_COMPATIBLE_VOICE_URL`), not a fork. No latency figure in this document was produced by a benchmark run — measure on the hardware that will serve patients.

TEST VERIFICATION: 445 Bun tests across 45 files • all provider calls mocked • oxlint & oxfmt clean • production build passes — Live validation required